Tema 6 — Mensajes y canales (Go)

Fuentes: teórica 07-mensajes, práctica 6 (clase-06-mensajes: Go básico, Victoria Canalizada, Pirámide), ejercicio extra "Streaming", labo de Go (`labo/labo-go`, partes y solución de la Pirámide), Guía 6. **En el parcial no entra teoría suelta: son ejercicios tipo guía**, así que este resumen está armado para resolver.

---

1. La idea

Hasta ahora todos los threads compartían memoria y peleábamos contra los interleavings. Con mensajes **no hay memoria compartida**: cada proceso tiene sus variables, y la única forma de interactuar es **mandar y recibir valores por canales**.

> "Don't communicate by sharing memory; share memory by communicating." — Rob Pike
>
> **En criollo:** en vez de una heladera compartida con candado (monitor), cada uno tiene su heladera y se mandan las cosas por correo. Si querés "algo compartido", ponés a **una persona** a cargo de esa heladera y le mandás pedidos. Como atiende de a un pedido por vez, la exclusión mutua sale gratis.

Un canal sirve para dos cosas a la vez: **pasar un dato** y **sincronizar** (el receive bloquea hasta que hay dato).

2. Sincrónico vs asincrónico ⭐

	Sincrónico (rendezvous)	**Asincrónico (con buffer)**
Go	`make(chan T)` = `make(chan T, 0)`	`make(chan T, n)`
Pseudocódigo	`create(channel T, 0)`	`create(channel T, k)` con k > 0
`send`	**bloquea hasta que alguien recibe**	bloquea solo si el buffer está lleno
`receive`	bloquea si no hay emisor	bloquea si el buffer está vacío
Qué te garantiza el send	que el otro **ya recibió** el dato	**nada** sobre el receptor: solo que entró al buffer

- En teoría, "asincrónico" = buffer **no acotado**. En Go siempre es acotado: el buffer absorbe ráfagas pero **conserva backpressure** (el productor se adelanta hasta `n` y después espera).
- Orden: los canales son **FIFO**. Lo que manda un mismo emisor por un mismo canal llega en orden.
- **Agregar buffer no arregla un protocolo mal hecho.** Si con `n = 0` hay deadlock, con `n = 5` probablemente lo haya igual, un rato después.

> **En criollo:** sincrónico es darle el paquete en mano (te quedás en la puerta hasta que te abren). Asincrónico es el buzón: lo tirás y te vas, pero si el buzón está lleno esperás.

Por qué importa para la guía: muchos ejercicios preguntan "¿sigue andando si la comunicación es asincrónica?" (Ej 6, 7b). La pregunta a hacerte es: **¿mi solución usaba en algún lado que "si el send volvió, el otro ya lo procesó"?** Si sí, en asincrónico se rompe y hace falta un **ack** (respuesta explícita).

3. Go mínimo (y su traducción al pseudocódigo de la guía)

La guía permite un pseudocódigo "tipo Go". Podés escribir Go real o esto; lo importante es que se entienda.

Concepto	Go	Pseudocódigo de la guía
lanzar un proceso	`go f(x)` / `go func(){ ... }()`	`process f(x)`
crear canal	`c := make(chan int)` / `make(chan int, k)`	`(channel int) c = create(channel int, k)`
enviar	`c <- v`	`send(c, v)`
recibir	`v := <-c`	`v = receive(c)`
cerrar	`close(c)`	`close(c)`
¿se cerró?	`v, ok := <-c` (`ok == false` → cerrado)	`receive` devuelve `EOF`
recorrer hasta el cierre	`for v := range c { ... }`	`while ((v = receive(c)) != EOF)`
canal nulo	`var c chan int` (vale `nil`)	`nil`
loop infinito / while	`for { }` / `for cond { }`	`while`
tipo unidad (señal sin dato)	`struct{}` y el valor `struct{}{}`	`unit`
"cualquier mensaje"	`any` (= `interface{}`) + `switch m := v.(type)`	`any` + switch de tipo
tiempo	`time.Sleep(d)`, `time.After(d)`, `time.NewTicker(d)`	`Sleep`, `After`, `Ticker`

Lo mínimo de Go que aparece en los ejemplos: - `x := 5` declara con inferencia (solo dentro de funciones). `var x int` arranca en el **valor cero** (0, "", false, `nil`). - El único loop es `for`. Sin paréntesis en `for` e `if`. - Structs: `type Req struct { X int; Reply chan int }`, literal `Req{X: 5, Reply: make(chan int)}`. **Se pasan por copia**, pero una copia de un canal es **el mismo canal**. - Slices: `buf = append(buf, v)`, `buf[0]`, `buf = buf[1:]`, `len(buf)`. Te sirven de cola dentro de un servidor. - `defer f()` corre `f` al salir de la función (típico: `defer wg.Done()`, `defer close(out)`). - **Direcciones de canal** (opcional pero prolijo): `chan<- T` solo envío, `<-chan T` solo recepción. El compilador te frena si usás mal un canal.

Detalles del pseudocódigo de la guía (leelos, cambian cosas)

- `main` **NO mata a los procesos al terminar** (en Go real sí). O sea: en la guía no necesitás un `WaitGroup` solo para que `main` espere.
- Un **servidor en loop infinito que queda esperando un mensaje para siempre no cuenta como deadlock.**
- **Siempre te piden justificar que no hay deadlock** (ver sección 9).

Goroutines

```
go trabajar(1)                      // función con nombre
go func(id int, c chan int) {       // anónima: pasale por parámetro lo que use
    c <- id * 2
}(i, c)
```

- `go` no espera ni devuelve nada. Para obtener un resultado, **que te lo mande por un canal**.
- En Go real, cuando `main` retorna se muere todo (por eso en Go real se usa `select {}` o un `WaitGroup` al final de `main`).

`close` y `range` ⭐

```
func produce(out chan<- int) {
    defer close(out)          // "no hay más"
    for i := 1; i <= 3; i++ { out <- i }
}
for v := range c { ... }      // termina cuando c se cierra y se vació
```

- `close` **no borra** lo que quedó en el buffer: el receptor lo recibe todo y recién ahí ve el cierre.
- Recibir de un canal cerrado **no bloquea**: devuelve el valor cero y `ok == false` (en la guía: `EOF`). **Siempre**, todas las veces.
- Enviar a un canal cerrado → **panic**. Cerrar dos veces → **panic**.
- **Cierra quien produce** (el único emisor, o el que sabe que todos los emisores terminaron). El receptor nunca cierra.
- Un `range` sobre un canal que nadie cierra **no termina nunca**.

`select` ⭐

```
select {
case v := <-c1:        // si hay algo en c1
    ...
case c2 <- x:          // si alguien puede recibir x en c2 (¡select también envía!)
    ...
case <-time.After(time.Second):
    ...                // timeout
default:
    ...                // si NINGUNO está listo ya: no bloquea
}
```

- Espera hasta que **algún** case pueda avanzar y ejecuta **exactamente uno**.
- Si hay varios listos, elige **al azar** (no por orden de escritura → **no hay prioridades**).
- Con `default` se vuelve **no bloqueante** (el "try").
- Elige **una vez**: para atender muchos mensajes, va **dentro de un `for`**.
- `select {}` bloquea para siempre.

Canal `nil`: apagar un case ⭐⭐

Enviar o recibir sobre `nil` **bloquea para siempre**, así que **un case con canal `nil` nunca se elige**. Es la forma de poner **guardas** (condiciones) en un `select`:

```
var entrada chan Pedido = pedidos
if lleno { entrada = nil }        // "mientras esté lleno no acepto pedidos"
select {
case p := <-entrada: ...          // apagado si entrada == nil
case r := <-respuestas: ...
}
```

Es **el reemplazo del `while (!cond) wait()`**: en vez de dormir al cliente adentro, el servidor **no le acepta el mensaje** hasta que pueda atenderlo. También sirve para **desactivar un canal que ya se cerró** (si no, el case de un canal cerrado está siempre listo y te come el `select` en un loop).

`sync.WaitGroup`

```
var wg sync.WaitGroup
for i := 0; i < n; i++ {
    wg.Add(1)                     // ANTES del go
    go func(i int) { defer wg.Done(); trabajar(i) }(i)
}
wg.Wait()                         // bloquea hasta que el contador vuelva a 0
```

- Se pasa **por puntero** (`*sync.WaitGroup`), nunca por copia.
- `Add` **antes** del `go`, no adentro de la goroutine (si no, `Wait` puede ver 0 antes de que arranque).
- Uso típico además de esperar en `main`: **saber cuándo terminaron todos los productores para recién ahí cerrar el canal** (fan-in, sección 6).

4. Errores que da Go (y qué significan)

- `fatal error: all goroutines are asleep - deadlock!` → todas bloqueadas (típico: send sincrónico sin receptor, o `range` sobre un canal que nadie cierra).
- `panic: send on closed channel` → alguien cerró antes de que terminaran los emisores (cerró el que no tenía que cerrar, o cerró antes de tiempo).
- `panic: close of closed channel` → dos procesos cierran el mismo canal.

5. Patrones de la teórica y la práctica (los ladrillos) ⭐

5.1 Servidor con estado (el "objeto" de los mensajes) ⭐⭐

Una goroutine dueña del estado, en un loop, atendiendo mensajes **de a uno**. Es el reemplazo de un monitor.

```
type Op interface{}                                  // "cualquier mensaje"
type Put struct{ F float64 }                         // fire and forget
type Get struct{ F float64; Reply chan float64 }     // con respuesta

func Cuenta(in chan Op) {
    bal := 0.0                                       // estado PRIVADO
    for {
        switch op := (<-in).(type) {
        case Put:
            bal += op.F
        case Get:
            bal -= op.F
            op.Reply <- bal
        }
    }
}
```

- Varios tipos de pedido → **un canal con `any` + type switch**, o **un canal por tipo de pedido + `select`** (con el segundo podés apagar tipos de pedido con `nil`).
- Si es un `string`/enum (`"Next"`, `"Count"`), alcanza con `switch cmd { case "Next": ... }`.

5.2 Pedido con canal de respuesta ⭐⭐

El canal viaja dentro del mensaje. Es la forma estándar de pedirle algo a un servidor y que la respuesta le llegue **a quien la pidió** y no a otro cliente.

```
type Req struct {
    X     int
    Reply chan int
}
// cliente
r := make(chan int, 1)            // buffer 1: el servidor nunca se traba al contestar
srv <- Req{X: 5, Reply: r}
y := <-r
// servidor
for { req := <-srv; req.Reply <- f(req.X) }
```

- **Un canal de respuestas compartido entre clientes está mal**: las respuestas se mezclan (la teórica lo muestra con el generador de números con tope distinto por cliente).
- Buffer 1 en el canal de respuesta = el servidor contesta y sigue aunque el cliente todavía no esté escuchando.
- **Separá canales por dirección.** Si usás el mismo canal con buffer para pedir y para recibir la respuesta, **podés leer tu propio pedido** (ejemplo de la teórica con P1/P2/P3). Con sincrónico no pasa, con buffer sí.

5.3 Delegar: una goroutine por pedido

```
for {
    req := <-pedidos
    go func(req Req) { atender(req) }(req)   // el servidor vuelve enseguida a recibir
}
```

Sirve cuando cada pedido es largo y no comparte estado con los otros (Ej 4c: una "sesión" por cliente).

5.4 Semáforo con canales

- **Directo:** un canal con buffer `n` lleno de tokens. `acquire` = `<-sem`, `release` = `sem <- struct{}{}`. (O al revés: canal vacío de capacidad `n`, `acquire` = mandar, `release` = recibir.)
- **Como servidor:** una goroutine con el contador `p` y canales `acq`, `rel`. Si `p == 0` **solo escucha** `rel`; si no, `select` entre los dos. Ojo: **acquire y release son los dos** `send` desde el cliente; el servidor elige cuál aceptar. Es el patrón "guarda" de la sección 3 escrito con `if` en vez de `nil`.

5.5 Rendezvous con confirmación (Victoria Canalizada)

Cada colaborador manda un `Pedido{id, terminado chan struct{}}` por `faros` o `carrocerias` y **se queda esperando en** `<-terminado`. Victoria recibe 1 carrocería y 2 faros (receives secuenciales, cada uno de su canal), arma, y después hace `terminado <- struct{}{}` a los tres. Es el pedido con canal de respuesta usado como **"avisame cuando termines"**.

5.6 Stream con cierre / productor–consumidor

`func produce() <-chan int` crea el canal, lanza una goroutine que manda y hace `defer close(out)`, y devuelve el canal. El consumidor hace `range`. (Parte I de la Pirámide: una goroutine expone los nombres por un canal y lo cierra al terminar; los que invitan hacen `nombre, ok := <-canalNombres` y cortan con `!ok`.)

5.7 Worker pool (fan-out / fan-in)

```
jobs, results := make(chan Job), make(chan Result)
for id := 0; id < W; id++ { go worker(jobs, results) }    // fan-out: W leen de jobs
go func() { defer close(jobs); for _, j := range input { jobs <- j } }()
for range input { fmt.Println(<-results) }                // fan-in: todos escriben en results

func worker(jobs <-chan Job, results chan<- Result) {
    for j := range jobs { results <- process(j) }         // termina cuando cierran jobs
}
```

- **Fan-out:** varios consumidores de un mismo canal → reparto automático de trabajo.
- **Fan-in:** varios emisores a un canal. **¿Quién cierra** `results`**?** Nadie de los workers (cerraría antes que los otros). Opciones: el que recibe sabe cuántos resultados esperar (como arriba), o una goroutine aparte hace `wg.Wait(); close(results)`.
- **El orden de los resultados no es el de los pedidos.** Si importa, ver 5.10.

5.8 Pipeline

Etapas encadenadas: cada una hace `for v := range in { out <- f(v) }; close(out)`. El cierre se propaga solo de una etapa a la siguiente. (Ej 9: el servicio T es una etapa "codificar y reenviar".)

5.9 Publicador / suscriptores (broadcast con registro dinámico)

El servidor tiene una lista (slice) de canales suscriptos y hace `select` entre: llega un dato → mandarlo a todos; llega una suscripción → `append`; llega una baja → sacarlo. (Ejercicio extra Streaming, Ej 8b de la guía.) - **Ojo con mandar a todos de forma sincrónica**: un suscriptor lento o que se fue traba al servidor entero. Usá canales con buffer, o mandá con `go func(){ ch <- x }()` si el orden entre envíos no importa. - Fin de la transmisión = el productor **cierra** su canal; la plataforma lo detecta con `ok == false` y cierra (o avisa) hacia abajo.

5.10 Respuestas en orden con varios servidores / pedidos en vuelo ⭐

Si hay varios pedidos en vuelo y al cliente le tenés que devolver las respuestas **en el orden en que pidió**: - A cada pedido reenviado le das **su propio canal de respuesta privado**. - Guardás esos canales en una **cola FIFO** (slice) en el orden de llegada. - Siempre esperás/entregás **la cabeza de la cola** (aunque otra respuesta posterior ya esté lista, esa queda en su canal con buffer hasta que le toque). - Para limitar a N en vuelo: contador + **guarda** `nil` sobre el canal de pedidos cuando llegás a N.

5.11 Timer / ticks

```
t := time.NewTicker(100 * time.Millisecond)
for {
    select {
    case <-t.C:            // cada tick
        ...
    case r := <-registro:  // y en el medio sigo atendiendo otras cosas
        ...
    }
}
```

`time.Sleep` en un loop alcanza si el proceso no tiene que atender nada más; si tiene que atender otros mensajes entre ticks, **Ticker +** `select`.

5.12 Prioridad (porque `select` no la tiene)

```
select {
case x := <-urgente: ...
default:
    select {
    case x := <-urgente: ...
    case x := <-normal: ...
    }
}
```

Primero miro solo el urgente sin bloquear; si no hay, espero cualquiera.

6. Terminación: quién cierra y cómo termina cada uno ⭐

Pregunta de diseño de la práctica, hacétela **para cada canal y cada goroutine**:

> ¿Quién produce, quién consume, **quién cierra** y **cómo termina** cada goroutine?

Situación	Cómo se termina
un productor, uno o más consumidores	el productor hace `close`; los consumidores hacen `range`
varios productores (fan-in)	`WaitGroup` de productores + una goroutine que hace `wg.Wait(); close(c)`, o el consumidor cuenta cuántos mensajes/"fin" espera
un emisor que se va de un servidor que le escucha	**mensaje centinela** ("me voy", pago de 0 en la Pirámide, `EOF`), o cerrar su canal propio
un servidor que tiene que dejar de escuchar un canal cerrado	`v, ok := <-c; if !ok { c = nil }` (si no, el case queda siempre listo)
servidor eterno	no termina; en la guía **no cuenta como deadlock**
"avisar que pasó algo" a muchos a la vez	`close(done)`: todos los que hacen `<-done` se desbloquean (un `send` despierta a uno solo)

7. Trampas (las que más importan)

1. **Send sincrónico sin nadie del otro lado = bloqueo.** En `main` de Go: `ch := make(chan int); ch <- 1` → deadlock inmediato.
2. **Ciclo de sends sincrónicos = deadlock.** A le manda a B mientras B le manda a A, y ninguno está en receive. Pasa en la Pirámide: el jefe manda comisiones al subordinado mientras el subordinado manda pagos al jefe. Arreglos: buffer en el canal, **mandar desde una goroutine aparte** (`go func(){ ch <- x }()`, lo que hace la solución del labo), o poner el send como case de un `select` que también recibe.
3. **Canal de respuesta compartido** entre clientes → la respuesta le llega a otro. Un canal de respuesta **por pedido** (o por cliente).
4. **Mismo canal con buffer en las dos direcciones** → te leés tu propio mensaje. Un canal por dirección.
5. **Cerrar desde el receptor, o cerrar con varios emisores vivos** → `panic: send on closed channel`.
6. `range` **sobre un canal que nadie cierra** → nunca termina.
7. **Canal cerrado en un** `select` **dentro de un loop** → ese case sale siempre (busy loop con valores cero). Ponerlo en `nil`.
8. **"Si el send volvió, el otro ya lo hizo"** solo vale con sincrónico. Con asincrónico, si necesitás saberlo: **ack**.
9. `select` **no es FIFO ni tiene prioridad**: elige al azar. Si te piden orden, guardalo vos (cola). Si te piden prioridad, select anidado (5.12).
10. **Servidor que se queda esperando la respuesta de un cliente** mientras otros clientes esperan al servidor → el servidor se vuelve secuencial o se traba. El servidor **no debería bloquearse en algo que depende de un cliente**: respuestas con buffer 1, o delegar en una goroutine.
11. **WaitGroup:** `Add` antes del `go`, pasarlo por puntero, `defer wg.Done()`.
12. **Variables del loop en una goroutine anónima:** pasalas como parámetro (`go func(i int){...}(i)`), como hace la teórica. Más claro y no depende de la versión de Go.
13. **Mandar un dato y después modificarlo:** si mandás un slice o un puntero, el receptor comparte memoria con vos (los slices son "vistas"). Mandá valores o copias.

8. Teoría mínima que sirve para justificar

- **Identificación indirecta:** los procesos no se nombran entre sí, se comunican por canales que se crean y se pasan por parámetro (o **por otros canales**). Los canales son "de primera clase".
- **Movilidad (π-cálculo):** que los canales viajen en mensajes hace que la topología cambie en ejecución. Es exactamente el patrón "pedido con canal de respuesta". (Solo para entender; no se toma.)
- **Exclusión mutua en un servidor:** el servidor procesa **un mensaje por vez** (es secuencial), así que su estado nunca se accede concurrentemente. No hace falta lock.
- **Concurrencia:** mandar todos los pedidos primero y recién después esperar todas las respuestas aprovecha la concurrencia; pedir-esperar-pedir-esperar la serializa (pregunta de la teórica con P3).

9. Cómo justificar "no hay deadlock" ⭐

En todos los ejercicios te lo piden. Un checklist: 1. **El servidor siempre vuelve a recibir.** Entre dos receives no hace nada que pueda bloquear para siempre (contestar por un canal con buffer, o a un cliente que seguro está esperando la respuesta). 2. **Todo cliente que espera una respuesta la va a recibir:** el servidor contesta todos los pedidos que acepta, por el canal de ese cliente. 3. **No hay ciclos de espera:** no hay A esperando a B mientras B espera a A (sends cruzados). Si los hay, están rotos con buffer, goroutine aparte o `select` que ofrece las dos cosas. 4. **Los `range` terminan** porque alguien cierra el canal (y ese alguien es el único emisor o espera a todos con un `WaitGroup`). 5. **Las guardas `nil` se vuelven a abrir:** si apagás un case por una condición, tiene que haber otro case activo que eventualmente cambie esa condición. 6. Un servidor esperando para siempre un pedido nuevo **no es deadlock** (regla de la guía).

10. Traducción desde lo que ya sabés

Monitores / semáforos / STM	Mensajes
estado compartido protegido por lock / `TVar`s	**variables locales de una goroutine servidor**
método `synchronized` / `atomically`	un **tipo de mensaje** que el servidor atiende en su loop (de a uno → atómico)
método que devuelve algo	mensaje con **canal de respuesta**
`while (!cond) wait(c)` / `check cond`	**no aceptar** ese pedido mientras no valga: guarda con canal `nil` en el `select`, o encolar el pedido y contestarlo más tarde
`signal` / `signalAll` / commit que despierta	el servidor **contesta** al/los cliente(s) encolados cuando cambia el estado
semáforo con `n` permisos	canal con buffer `n` lleno de tokens, o servidor con contador
`acquire(k)` atómico (no se puede con semáforos)	**sí se puede**: el servidor acepta el pedido de `k` solo si hay `k`
`orElse` (preferencia)	`select` (pero **al azar**, no por preferencia); preferencia real → `select` anidado con `default`
`tryAcquire` / versión no bloqueante	`select` con `default`
ticket FIFO para no tener inanición	**cola (slice) de pedidos pendientes** en el servidor, atendida en orden
barrera / esperar a n	contar n mensajes "llegué" y después mandar n "seguí" (o `close(done)`)
`join` / `wait` de `async`	`WaitGroup`, o recibir un "terminé" por un canal
`while (true)` del thread	`for { }`

> **En criollo:** todo lo que hacías con un monitor, ahora lo hace un proceso servidor. Los "dormidos en la condición" son **pedidos que el servidor todavía no contestó** (o que todavía no le recibió).

11. Esqueletos reutilizables ⭐

Servidor con varios tipos de pedido y respuestas

```
type Pedir struct{ Cant int; Reply chan bool }
type Devolver struct{ Cant int }

func Servidor(pedir chan Pedir, devolver chan Devolver) {
    estado := 0                        // lo que sea
    for {
        select {
        case p := <-pedir:
            // decidir, actualizar estado, contestar
            p.Reply <- true
        case d := <-devolver:
            // actualizar estado (fire and forget)
        }
    }
}
```

Servidor con guardas (pedidos que "esperan una condición")

```
for {
    var entrada chan Pedido            // nil = apagado
    if puedoAtender(estado) { entrada = pedidos }
    select {
    case p := <-entrada:               // solo si la condición vale
        ...
    case x := <-otroCanal:             // esto es lo que hace cambiar la condición
        ...
    }
}
```

Variante cuando la condición depende **del pedido** (no sé si lo puedo atender hasta leerlo): recibir siempre, y si no se puede, **encolarlo** en un slice de pendientes; después de cada cambio de estado, recorrer la cola y contestar los que ahora sí se pueden (en orden si piden fairness).

Cola interna con guardas de entrada y salida

```
var cola []T
for {
    var sal chan T; var prox T
    if len(cola) > 0 { sal = salida; prox = cola[0] }  // solo ofrezco salir si hay algo
    var ent chan T = entrada
    if len(cola) == MAX { ent = nil }                  // (si es acotada)
    select {
    case v := <-ent:   cola = append(cola, v)
    case sal <- prox:  cola = cola[1:]
    }
}
```

Notar el `case sal <- prox`: **el select también ofrece enviar**.

`main` del sistema (pseudocódigo de la guía, sin WaitGroup)

```
(channel Cmd) ch = create(channel Cmd, 0)
process Servidor(ch)
process Cliente(3, ch)
process Cliente(5, ch)
```

En Go real agregá `select {}` al final de `main` (o un `WaitGroup`) para que no se muera todo.

12. Patrones para la Guía 6

Una línea por ejercicio: la idea, no la solución.

Ej	Patrón / técnica
1	**Servidor con estado** (contador de `Next`), un solo canal compartido por los dos clientes, `switch` sobre el comando. Clientes: loop infinito de n sends + 1 send.
2	**Pedido con canal de respuesta**: `Count` pasa a ser un struct con `Reply`. Cada cliente usa **su** canal. Pensar si conviene buffer 1.
3	Servidor que recibe **de dos canales en pareja**: un receive de cada uno por vuelta (alcanza con dos receives secuenciales; pensar qué pasa con los clientes mientras tanto según el canal sea sincrónico o no).
4a	**Servidor con estado + respuesta** (acertó/no) hasta que el cliente acierta.
4b	**Sesiones secuenciales**: el servidor atiende un cliente entero antes de aceptar el siguiente (el cliente entra mandando su canal de respuesta; los demás quedan esperando en el canal de entrada).
4c	**Una goroutine por cliente** (5.3): el servidor solo reparte "sesiones", cada una con su número y su canal.
5	**Cola interna + `select` con guarda `nil`** sobre el envío (esqueleto "cola interna", sin tope). Pensar qué hacer si cierran la entrada. ☆
6	Asincrónico con pedidos en vuelo: **cada respuesta tiene que identificar a qué pedido contesta** (canal de respuesta por pedido, o incluir el número intentado en la respuesta).
7a	**Servidor = variable compartida**: mensajes `Read{Reply}` y `Write{v}`.
7b	Pregunta del tipo "¿qué garantizaba el sincrónico?": con asincrónico el `Write` no confirma nada → **ack** si hace falta que el que escribió sepa que ya impactó (y pensar el orden entre clientes).
8a	**Timer**: loop con `Sleep` / `Ticker` mandando `struct{}{}` a cada canal del slice. Asincrónico → no se traba con un cliente lento.
8b	**Registro dinámico**: `select` entre el tick y el canal de registro (`append`). Mismo patrón que Streaming b/c.
9a	**Etapa de pipeline**: recibir de C, `codificar`, reenviar a R.
9b	**Mensajes de control intercalados con datos**: dos tipos de mensaje (cambio de destino / texto) por el mismo canal para conservar el orden; el servicio guarda "destino actual" como estado.
9c	**Pedido con canal de respuesta a K** + **pedir la clave por adelantado** (prefetch de a una, con canal propio con buffer) para no esperar después de cada mensaje sin acumular pedidos innecesarios a K.
10a	**Proxy que serializa**: reenvía un pedido a S con un canal de respuesta propio, espera, reenvía al cliente, recién ahí acepta el siguiente.
10b	**Hasta N en vuelo**: contador + guarda `nil` sobre la entrada cuando hay N; como S responde en orden, una **cola FIFO** de canales de clientes alcanza (5.10). ☆
10c	**Dos servidores + orden de respuestas**: repartir entre las dos copias y devolver en el orden pedido con **cola FIFO de canales de respuesta privados** (5.10). ☆

Extras que no están en la guía pero entrenan lo mismo: - **Streaming** (ejercicio extra): pub/sub (5.9). a) broadcast a lista fija; b) registro inicial de 3 + avisar al streamer que arranque; c) altas/bajas durante la transmisión; d) cuando quedan 0 suscriptores, avisar al streamer, que **cierra** su canal (terminación por `close`). - **Victoria Canalizada** (práctica): rendezvous con confirmación (5.5). - **Pirámide** (labo): árbol de goroutines con `WaitGroup` (Add antes de `go`), stream de nombres con cierre, `select` + `Ticker`, centinela de fin (pago 0), canales en `nil` para dar de baja subordinados, y el **deadlock por sends cruzados** jefe ↔ subordinado resuelto con buffer 1 + `go func(){ ch <- x }()` (trampa 2).

13. Plantilla mental para cualquier ejercicio de mensajes

1. **¿Quiénes son los procesos?** Dibujá cajitas y flechas: cada flecha es un canal. ¿Quién crea cada canal y quién se lo pasa a quién?
2. **¿Qué estado hay y quién es el dueño?** Cada cosa "compartida" vive en **un** proceso servidor.
3. **¿Qué mensajes hay?** Un struct por tipo. Si esperan algo de vuelta → **canal de respuesta adentro**.
4. **¿Sincrónico o asincrónico?** (Lo suele decir el enunciado.) ¿Mi protocolo depende de que el send implique recepción?
5. **¿Hay pedidos que no se pueden atender todavía?** → guarda `nil` o cola de pendientes.
6. **¿Importa el orden / la fairness?** → cola FIFO propia (el `select` es al azar).
7. **Terminación:** quién cierra cada canal, cómo termina cada goroutine.
8. **Justificar no deadlock** con el checklist de la sección 9.